Теория вероятностей в Data Science на примере данных о "Титанике"

Цель. Исследуем фундаментальные понятия теории вероятностей, используя данные о пассажирах "Титаника". Ответим на конкретные вопросы, используя: условную вероятность, формулу полной вероятности, теорему Байеса; шаг за шагом разбирем, как условная вероятность, формула полной вероятности и теорема Байеса работают на реальных данных

Источник данных: https://www.kaggle.com/competitions/titanic/data?select=train.csv (https://www.kaggle.com/competitions/titanic/data?select=train.csv)



Импорт библиотек

Ввод [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Загрузка данных и их описание

Ввод [2]:

```
# Загружаем данные
df = pd.read_csv('Data/train.csv')   # Обучающая выборка train.csv с Kaggle

print("Размерность данных:", df.shape)
```

```
Размерность данных: (891, 12)
```

Ввод [3]:

```
# Быстрый взгляд на данные
df.head()
```

Out[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN

Описание признаков набора данных

Ввод [4]:

```
print("\nСтолбцы:", df.columns.tolist())
```

```
Столбцы: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'P
arch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
```

Ввод [5]:

```
# Информация о данных
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Признак	Тип	Описание	Проблемы	Потенциал для Feature Engineering
`PassengerId`	Числовой	Уникальный ID	Не информативен	Удаление
`Survived`	Бинарный (выжил (1) или нет (0))	Целевая переменная	–	–
`Pclass`	Категор. порядк.	Класс билета (`1` =1-й, `2` =2-й, `3` =3-й)	–	–
`Name`	Текстовый	Имя пассажира	–	Извлечение титула
`Sex`	Категор.	Пол	–	Кодировка (*Label Encoding*)
`Age`	Числовой	Возраст	Пропуски	Создание возрастных групп
`SibSp`	Числовой	Количество братьев/сестер/супругов на борту	–	Объединение с `Parch`
`Parch`	Числовой	Количество родителей/детей на борту	–	Объединение с `SibSp`
`Ticket`	Текстовый	Номер билета	Сложная структура	Извлечение префикса
`Fare`	Числовой	Стоимость билета	Выбросы	Логарифмирование, бининг
`Cabin`	Текстовый	Номер каюты	Много пропусков	Извлечение палубы (`Deck`)
`Embarked`	Категор.	Порт посадки (`C` =Шербур, `Q` =Квинстаун, `S` =Саутгемптон)	Несколько пропусков	–

Ввод [6]:

```
# Пропущенные значения
print("Пропущенные значения:")
missing_data = df.isnull().sum()
missing_percent = (missing_data / len(df)) * 100
pd.DataFrame({'Количество пропусков': missing_data,
              'Процент пропусков': missing_percent.round(2)})
```

```
Пропущенные значения:
```

Out[6]:

	Количество пропусков	Процент пропусков
PassengerId	0	0.00
Survived	0	0.00
Pclass	0	0.00
Name	0	0.00
Sex	0	0.00
Age	177	19.87
SibSp	0	0.00
Parch	0	0.00
Ticket	0	0.00
Fare	0	0.00
Cabin	687	77.10
Embarked	2	0.22

Создадим сводную таблицу для первоначального взгляда

Ввод [6]:

```
print("Доля выживших по классу и полу:")
pivot = pd.pivot_table(
    df,
    values='Survived',
    index=['Pclass'],
    columns=['Sex'],
    aggfunc= 'mean',
    fill_value=0
)
pivot.round(4)
```

```
Доля выживших по классу и полу:
```

Out[6]:

Sex	female	male
Pclass		
1	0.9681	0.3689
2	0.9211	0.1574
3	0.5000	0.1354

Вероятности и условные вероятности

Простые вероятности

Вычислим некоторые априорные (*prior*) вероятности.

Ввод [7]:
```
# P(Выжил) - общая вероятность выживания
P_Survived = df['Survived'].mean()
print(f"P(Survived) = {P_Survived:.3f}")
```

```
P(Survived) = 0.384
```

Ввод [8]:
```
# P(Класс) - вероятность быть пассажиром каждого класса
for pclass in sorted(df['Pclass'].unique()):
    P_Class = (df['Pclass']==pclass).mean()
    print(f"P(Pclass={pclass}) = {P_Class:.3f}")
```

```
P(Pclass=1) = 0.242
P(Pclass=2) = 0.207
P(Pclass=3) = 0.551
```

Ввод [9]:
```
# P(Пол) - вероятность быть мужчиной или женщиной
for sex in df['Sex'].unique():
    P_Sex = (df['Sex']==sex).mean()
    print(f"P(Sex={sex}) = {P_Sex:.3f}")
```

```
P(Sex=male) = 0.648
P(Sex=female) = 0.352
```

Условная вероятность $P_B(A)$

Задача 1.

Какова вероятность выжить, если пассажир был женщиной? $P_{Female}(Survived)$

Формула:

$$P_{Female}(Survived) = \frac{P(Survived \cdot Female)}{P(Female)}$$

Ввод [10]:
```
# Способ 1: Вручную по формуле
women = df[df['Sex']=='female']
P_Female = (df['Sex']=='female').mean()
P_Survived_and_Female = ((df['Survived']==1) & (df['Sex']=='female')).mean()

P_Survived_if_Female = P_Survived_and_Female / P_Female
print(f"P(Survived | Female), рассчитанная вручную:
      {P_Survived_if_Female:.3f}")
```

```
P(Survived | Female), рассчитанная вручную: 0.742
```

Ввод [11]:
```
# Способ 2: Просто отфильтровав датафрейм и посчитав среднее
women = df[df['Sex']=='female']
P_P_Survived_if_Female = women['Survived'].mean()
print(f"P(Survived | Female), рассчитанная через фильтр:
      {P_P_Survived_if_Female:.3f}")
```

```
P(Survived | Female), рассчитанная через фильтр: 0.742
```

Задача 2.

Какова вероятность выжить, если пассажир был мужчиной? $P_{Male}(Survived)$

Формула:

$$P_{Male}(Survived) = \frac{P(Survived \cdot Male)}{P(Male)}$$

Ввод [13]:
```
# Способ 1: Вручную по формуле
men = df[df['Sex']=='male']
P_Male = (df['Sex']=='male').mean()
P_Survived_and_Male = ((df['Survived']==1) & (df['Sex']=='male')).mean()

P_Survived_if_Male = P_Survived_and_Male / P_Male
print(f"P(Survived | Male), рассчитанная вручную: {P_Survived_if_Male:.3f}")
```

```
P(Survived | Male), рассчитанная вручную: 0.189
```

Ввод [14]:
```
# Способ 2: Просто отфильтровав датафрейм и посчитав среднее
men = df[df['Sex']=='male']
P_P_Survived_if_Male = men['Survived'].mean()
print(f"P(Survived | Male), рассчитанная через фильтр:
      {P_P_Survived_if_Male:.3f}")
```

```
P(Survived | Male), рассчитанная через фильтр: 0.189
```

Задача 3.

Какова вероятность выжить, если пассажир был из 3-го класса? $P_{Pclass=3}(Survived)$

Формула:

$$P_{Pclass=3}(Survived) = \frac{P(Survived \cdot Pclass = 3)}{P(Pclass = 3)}$$

Ввод [15]:
```
# Способ 1: Вручную по формуле
class3 = df[df['Pclass']==3]
P_Pclass3 = (df['Pclass']==3).mean()
P_Survived_and_Pclass3 = ((df['Survived']==1) & (df['Pclass']==3)).mean()

P_Survived_if_Pclass3 = P_Survived_and_Pclass3 /P_Pclass3
print(f"P(Survived | Pclass3), рассчитанная вручную:
      {P_Survived_if_Pclass3:.3f}")
```

```
P(Survived | Pclass3), рассчитанная вручную: 0.242
```

Ввод [16]:

```
# Способ 2: Просто отфильтровав датафрейм и посчитав среднее
class3 = df[df['Pclass']==3]
P_P_Survived_if_Pclass3 = class3['Survived'].mean()
print(f"P(Survived | Pclass), рассчитанная через фильтр:
      {P_P_Survived_if_Pclass3:.3f}")
```

```
P(Survived | Pclass), рассчитанная через фильтр: 0.242
```

Обобщение вычисления условных вероятностей

Поскольку при вычислении условных вероятностей прослеживается общая схема их расчета, напишем **функцию определения условных вероятностей**, которая будет принимать:

- `df` – датафрейм;
- `condition_col` – название столбца с произошедшим событием;
- `condition_val` – значение произошедшего события;
- `target_col` – название столбца с целевым событием;
- `target_val` – значение целевого события.

Ввод [17]:

```
def conditional_probability(df, condition_col, condition_val,
                            target_col, target_val=1):
    """
    Вычисляет условную вероятность
    P(target_col=target_val | condition_col=condition_val)
    """
    subset = df[df[condition_col]==condition_val]
    if len(subset) == 0:
        return 0
    return (subset[target_col]==target_val).mean()
```

Ввод [18]:

```
print("УСЛОВНЫЕ ВЕРОЯТНОСТИ ВЫЖИВАНИЯ:")
print("=" * 50)

# По полу
print("\nПо полу:")
for sex in df['Sex'].unique():
    prob = conditional_probability(df, 'Sex', sex, 'Survived')
    print(f"  P(Survived|Sex={sex}) = {prob:.3f}")

# По классу
print("\nПо классу:")
for pclass in sorted(df['Pclass'].unique()):
    prob = conditional_probability(df, 'Pclass', pclass, 'Survived')
    print(f"  P(Survived|Pclass={pclass}) = {prob:.3f}")

# По порту посадки
print("\nПо порту посадки:")
for embarked in sorted(df['Embarked'].dropna().unique()):
    prob = conditional_probability(df, 'Embarked', embarked, 'Survived')
    print(f"  P(Survived|Embarked={embarked}) = {prob:.3f}")
```

```
УСЛОВНЫЕ ВЕРОЯТНОСТИ ВЫЖИВАНИЯ:
==================================================

По полу:
  P(Survived|Sex=male) = 0.189
  P(Survived|Sex=female) = 0.742

По классу:
  P(Survived|Pclass=1) = 0.630
  P(Survived|Pclass=2) = 0.473
  P(Survived|Pclass=3) = 0.242

По порту посадки:
  P(Survived|Embarked=C) = 0.554
  P(Survived|Embarked=Q) = 0.390
  P(Survived|Embarked=S) = 0.337
```

Вывод. Условная вероятность позволяет нам уточнить наши прогнозы, учитывая новую информацию (пол пассажира).

Формула полной вероятности

Задача. Мы знаем вероятность выживания для каждого класса. Как, используя эту информацию, найти общую вероятность выживания $P(Survived)$?

Формула полной вероятности:

$P(A) = \sum P_{B_i}(A) \cdot P(B_i)$, где B_i образуют полную группу несовместных событий.

В нашем случае: $\{B_1, B_2, B_3\} = \{Pclass = 1, Pclass = 2, Pclass = 3\}$

$$P(Survived) = P(Survived|Pclass = 1) \cdot P(Pclass = 1) + P(Survived|Pclass = 2) \cdot P(Pclas$$
$$+ P(Survived|Pclass = 3) \cdot P(Pclass = 3)$$

Ввод [19]:

```
print("ФОРМУЛА ПОЛНОЙ ВЕРОЯТНОСТИ:")
print("P(Survived) = Σ P(Survived|Pclass=i) * P(Pclass=i)")
print("=" * 50)

total_prob = 0
prob_details = []

for pclass in sorted(df['Pclass'].unique()):
    # P(Pclass = i)
    P_class = (df['Pclass']==pclass).mean()
    # P(Survived | Pclass = i)
    P_survived_given_class = conditional_probability(df, 'Pclass',
                                                     pclass, 'Survived')

    # Вклад этого класса
    contribution = P_survived_given_class * P_class
    total_prob += contribution

    prob_details.append({
        'Class': pclass,
        'P(Class)': P_class,
        'P(Survived|Class)': P_survived_given_class,
        'Вклад': contribution
    })

# Создаем DataFrame для наглядности
prob_df = pd.DataFrame(prob_details)
print(prob_df.round(3))

print(f"\nОбщая вероятность выживания, рассчитанная по формуле полной вероятност
      {total_prob:.3f}")
print(f"Общая вероятность выживания, рассчитанная напрямую из данных:
      {df['Survived'].mean():.3f}")
```

```
ФОРМУЛА ПОЛНОЙ ВЕРОЯТНОСТИ:
P(Survived) = Σ P(Survived|Pclass=i) * P(Pclass=i)
==================================================
   Class  P(Class)  P(Survived|Class)  Вклад
0      1     0.242              0.630  0.153
1      2     0.207              0.473  0.098
2      3     0.551              0.242  0.134

Общая вероятность выживания, рассчитанная по формуле полной вероятности: 0.384
Общая вероятность выживания, рассчитанная напрямую из данных: 0.384
```

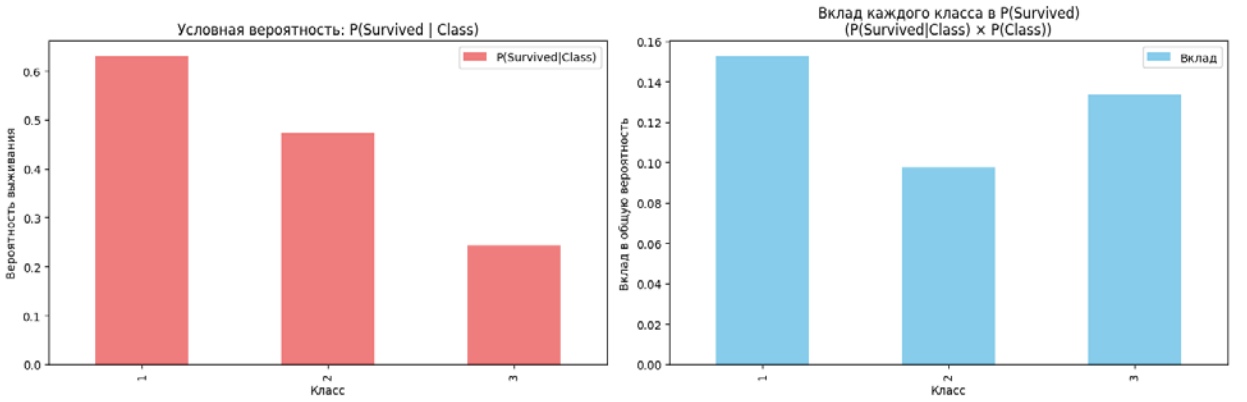
Ввод [20]:

```
# Визуализируем вклад классов
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Левая часть: P(Survived | Class)
prob_df.plot(x='Class', y='P(Survived|Class)', kind='bar',
             ax=ax1, color='lightcoral')
ax1.set_title('Условная вероятность: P(Survived | Class)')
ax1.set_ylabel('Вероятность выживания')
ax1.set_xlabel('Класс')

# Правая часть: Вклад в общую вероятность
prob_df.plot(x='Class', y='Вклад', kind='bar', ax=ax2, color='skyblue')
ax2.set_title('Вклад каждого класса в P(Survived)\n(P(Survived|Class) × P(Class)')
ax2.set_ylabel('Вклад в общую вероятность')
ax2.set_xlabel('Класс')

plt.tight_layout()
plt.show()
```



Вывод. Формула полной вероятности позволяет вычислить вероятность сложного события (Выжил), разбив его на сценарии (Класс), по которым нам проще считать.

Формула Байеса (формула проверки гипотез)

Задача. Мы видим выжившего пассажира. Информации о его поле у нас нет, но мы хотим понять, из какого он, скорее всего, класса. Другими словами, мы хотим найти $P_{Survived}(Pclass)$.

Теорема Байеса позволяет это найти:

$$P_B(A) = \frac{P(A) \cdot P_A(B)}{P(B)}$$

В нашем случае:

- `A : Pclass=i ;`
- `B : Survived=1 .`

$$P_{Survived}(Pclass = i) = \frac{P(Pclass = i) \cdot P_{Pclass=i}(Survived)}{P(Survived)}$$

Ввод [21]:

```
print("ТЕОРЕМА БАЙЕСА:")
print("P(Pclass | Survived) = [P(Survived | Pclass) × P(Pclass)] / P(Survived)")
print("=" * 50)

# P(Survived) - знаменатель
P_S = df['Survived'].mean()

bayes_results = []

for pclass in sorted(df['Pclass'].unique()):
    # P(Pclass) - априорная вероятность
    P_A = (df['Pclass']==pclass).mean()
    # P(Survived | Pclass) - правдоподобие
    P_B_given_A = conditional_probability(df, 'Pclass', pclass, 'Survived')

    # Применяем теорему Байеса
    P_A_given_B = (P_B_given_A * P_A) / P_S

    bayes_results.append({
        'Class': pclass,
        'P(Class)': P_A,
        'P(Survived|Class)': P_B_given_A,
        'P(Class|Survived)': P_A_given_B
    })

# Создаем DataFrame
bayes_df = pd.DataFrame(bayes_results)
print("Результаты применения теоремы Байеса:")
print(bayes_df.round(3))
```

```
ТЕОРЕМА БАЙЕСА:
P(Pclass | Survived) = [P(Survived | Pclass) × P(Pclass)] / P(Survived)
==================================================
Результаты применения теоремы Байеса:
   Class  P(Class)  P(Survived|Class)  P(Class|Survived)
0      1     0.242              0.630              0.398
1      2     0.207              0.473              0.254
2      3     0.551              0.242              0.348
```

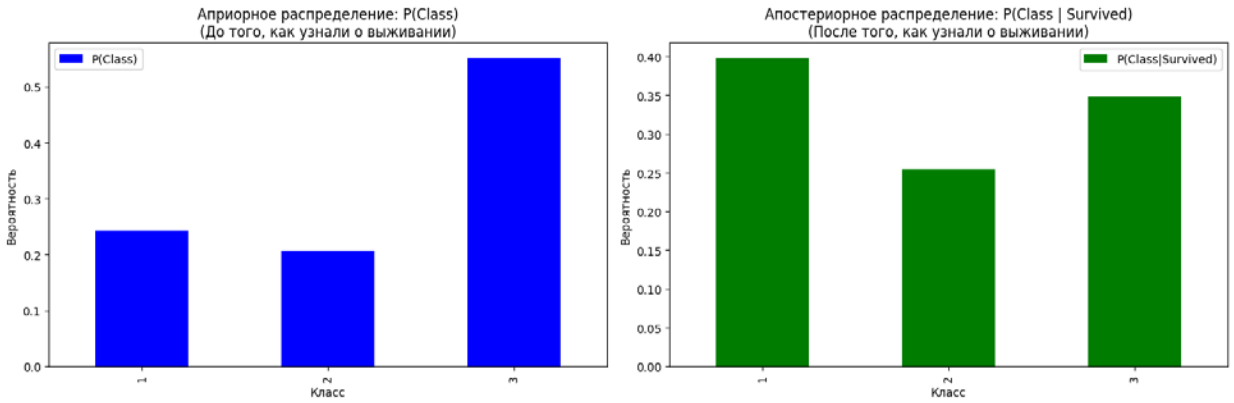
Ввод [22]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Априорное распределение
bayes_df.plot(x='Class', y='P(Class)', kind='bar', ax=ax1, color='blue')
ax1.set_title('Априорное распределение: P(Class)\n(До того, как узнали о выжива
ax1.set_ylabel('Вероятность')
ax1.set_xlabel('Класс')

# Апостериорное распределение
bayes_df.plot(x='Class', y='P(Class|Survived)', kind='bar', ax=ax2, color='gree
ax2.set_title('Апостериорное распределение: P(Class | Survived)\n(После того, к
ax2.set_ylabel('Вероятность')
ax2.set_xlabel('Класс')

plt.tight_layout()
plt.show()
```



Вывод. Теорема Байеса позволяет обновлять наши убеждения (о классе пассажира) на основе новых свидетельств (факт выживания). Мы видим, что среди выживших доля пассажиров 1-го и 2-го класса значительно выше, чем их общая доля на корабле.

Домашнее задание

Выберите самостоятельно некоторый набор данных (можно с *Kaggle*), проанализируйте и опишите признаки. На основе выбранного набора покажите применение условных вероятностей, формулы полной вероятности и теоремы Байеса. Сделайте выводы о том, как вероятностные методы помогают извлекать инсайты и строить прогнозы в реальных бизнес- и научных задачах!

Можно использовать следующие датасеты:

№ п/п	Название набора	Краткое описание
1.	`Heart Disease Prediction`	Предсказание сердечно-сосудистых заболеваний
2.	`Diabetes Prediction Dataset`	Диагностика диабета
3.	`Breast Cancer Wisconsin`	Диагностика рака груди
4.	`COVID-19 Symptoms and Risk Factors`	COVID-19 и факторы риска
5.	`Medical Insurance Charges`	Страховые выплаты
6.	`Credit Card Fraud Detection`	Обнаружение мошенничества
7.	`Loan Prediction Dataset`	Прогнозирование выдачи кредита
8.	`Bank Marketing Dataset`	Эффективность маркетинга банка

№ п/п	Название набора	Краткое описание
9.	Credit Risk Dataset	Оценка кредитного риска
10.	Online Shopper Purchasing Intention	Намерение покупки онлайн
11.	Customer Churn Prediction	Предсказание оттока клиентов
12.	Amazon Product Reviews	Анализ отзывов
13.	Retail Transaction Data	Транзакционные данные
14.	Telco Customer Churn	Отток в телекоме
15.	Network Intrusion Detection	Обнаружение сетевых атак
16.	Software Defect Prediction	Предсказание дефектов ПО
17.	Students Performance Dataset	Успеваемость студентов
18.	HR Analytics Dataset	Анализ текучести кадров
19.	Census Income Dataset	Предсказание дохода
20.	Crime Classification	Классификация преступлений
21.	Wine Quality Dataset	Качество вина
22.	Airline Passenger Satisfaction	Удовлетворенность авиапассажиров
23.	Car Insurance Claim Prediction	Страховые случаи авто